

Internal Working of pop() in Python List

Before understanding this method

A Python list works like a dynamic array. Internally it has two important ideas:

- **size** - number of elements currently present in the list.
- **capacity** - extra internal space reserved by Python for future elements.

When capacity is full, Python creates a bigger internal array, copies old elements, and then performs the operation.

Meaning

pop() removes an element from the list and returns it. By default, it removes the last element.

```
a = [10, 20, 30, 40]
x = a.pop()
print(a)    # [10, 20, 30]
print(x)    # 40
```

Internal working of pop() without index

- Python directly removes the last element.
- No shifting is needed because the last element is removed.
- The removed value is returned.
- The size of the list decreases by 1.

```
Before pop():
Index:  0   1   2   3
List:  10  20  30  40
```

```
After pop():
Removed = 40
List:  10  20  30
```

Internal working of pop(index)

When an index is given, Python removes the element at that index and shifts later elements left.

```
a = [10, 20, 30, 40, 50]
x = a.pop(2)
print(a)    # [10, 20, 40, 50]
print(x)    # 30
```

```
Before pop(2):
Index:  0   1   2   3   4
List:  10  20  30  40  50
```

```
Remove 30 and shift left:
40 moves from index 3 to 2
50 moves from index 4 to 3
```

```
Final list:
[10, 20, 40, 50]
```

Time complexity

- `pop()` from end: $O(1)$.
- `pop(index)` from beginning or middle: $O(n)$, because shifting is required.

Key point

`pop()` removes by index and returns the removed value.